

Competitive Programming

Section 56: Hard Greedy And Matroids Intro

Competitive Programming Course

2026-07-10

Table of contents

1	Exchange arguments at scale	3
1.1	Exchange argument template	3
1.2	Deadline profit	3
1.3	Minimum interval cover	3
1.4	Kruskal as matroid greedy	4
1.5	When greedy is suspicious	4
1.6	Deadline scheduling trace	4
1.7	Interval cover trace	4
1.8	Kruskal proof sketch	5
1.9	Counterexample habit	5
1.10	Proof writing pattern	5
1.11	Matroid intuition without formalism	5
2	Exercises	6
2.1	Proof-first greedy workshop	6

1 Exchange arguments at scale

Matroid-like greedy works when local exchanges preserve feasibility. Deadline scheduling keeps the best profits among jobs that can fit so far; interval cover greedily extends the covered prefix; Kruskal keeps edges that preserve forest independence.

1.1 Exchange argument template

A greedy proof often compares your solution G with an optimal solution O . Show that if O does not make the greedy choice, you can exchange one part of O for the greedy choice without making it worse.

Template:

```
Let  $g$  be the greedy choice.  
Take an optimal solution  $O$ .  
If  $O$  already contains  $g$ , continue.  
Otherwise replace some compatible element of  $O$  by  $g$ .  
The replacement remains feasible and is no worse.  
Therefore some optimal solution agrees with greedy.
```

1.2 Deadline profit

For unit-time jobs with deadlines and profits:

1. Sort jobs by deadline.
2. Add each job's profit to a min-heap.
3. If the heap size exceeds the current deadline, remove the smallest profit.

Invariant: after processing jobs up to deadline d , the heap contains the best set of jobs that can fit in d slots.

The exchange is direct: if too many jobs are selected, dropping the smallest profit is at least as good as dropping any other selected job.

1.3 Minimum interval cover

To cover $[0, T]$, maintain the leftmost uncovered coordinate cur . Among all intervals with $l \leq cur$, choose the one with maximum r .

This is optimal because every valid solution must choose some interval starting at or before cur ; choosing the one that reaches farthest leaves a suffix that is no harder to cover.

1.4 Kruskal as matroid greedy

In MST, feasible edge sets are forests. Kruskal processes edges by increasing weight and adds an edge if it connects two different components.

The exchange argument: for any cut, the lightest edge crossing that cut can be included in some MST. DSU enforces the forest constraint efficiently.

1.5 When greedy is suspicious

Ask:

- What is the feasibility structure?
- Can a local choice be exchanged into an optimal solution?
- Does sorting by one key ignore a second constraint?
- Can I generate a small counterexample?

If you cannot answer these, use DP, flow, or search instead of forcing greedy.

1.6 Deadline scheduling trace

Jobs:

```
(deadline, profit): (1,10), (1,5), (2,7)
```

Process by deadline:

```
job (1,10): heap=[10]
job (1,5):  heap=[5,10], size 2 > deadline 1, remove 5 -> [10]
job (2,7):  heap=[7,10]
```

Best profit is 17. The removed job is safe to discard because among jobs that compete for the same limited slots, the smallest profit is the least valuable.

1.7 Interval cover trace

Cover $[0,10]$ with intervals:

```
[0,4], [0,6], [4,10], [6,10]
```

At $cur=0$, choose $[0,6]$ because it reaches farthest. At $cur=6$, choose $[6,10]$. Two intervals cover the target. Choosing $[0,4]$ first cannot lead to a better solution because it leaves a strictly harder suffix.

1.8 Kruskal proof sketch

When Kruskal considers an edge e connecting two different DSU components, look at the cut between those components. Any spanning tree must include some edge crossing that cut. Since Kruskal processes edges by increasing weight, e is no heavier than the crossing edge used by an optimal tree. Exchanging that edge for e preserves connectivity and does not increase cost.

This is the cut property. It is the reason the DSU check is enough.

1.9 Counterexample habit

Before trusting a greedy rule, try to break it with three or four objects.

For deadline jobs, “take highest profit first” can fail if a high-profit late job crowds out several earlier feasible jobs. Sorting by deadline and keeping a heap survives this because it maintains feasibility at every prefix of deadlines.

For interval cover, “take shortest interval” and “take earliest ending interval” both fail. The correct local choice is farthest reach among intervals that start before the current uncovered point.

1.10 Proof writing pattern

A complete greedy proof should contain:

```
Greedy choice:  
Feasibility after choosing it:  
Exchange argument or cut property:  
Why the remaining subproblem has the same form:
```

If one of these lines is missing, the solution may still be intuitive, but the editorial is not yet complete.

1.11 Matroid intuition without formalism

A matroid is a setting where “independent” sets support greedy choice by weight. Forests in a graph are the standard example: adding an edge that does not create a cycle preserves independence. That is why Kruskal works.

Most contest greedy problems are not presented as matroids, but the exchange habit is the same. If feasible sets do not support exchanges, greedy is dangerous.

2 Exercises

2.1 Proof-first greedy workshop

For a greedy claim, write the selected object, the first differing choice of an optimal solution, and the exchange that repairs it. In deadline profit, keep a min-heap of chosen profits: when a deadline allows d jobs, discard the smallest profit if more than d jobs are selected. In interval cover, among intervals starting before the uncovered point choose the farthest right endpoint. In MST, the cut property says a lightest crossing edge is safe. Test equal deadlines, gaps in coverage, duplicate weights, and a tempting locally cheap edge that creates a cycle. Run `CP_TARGET=solution python3 sections/56_hard_greedy_matroids_intro/check.py`.

- A. Deadline Profit: Each unit job has deadline and profit. Maximize profit.
- B. Minimum Interval Cover: Minimum intervals needed to cover $[0, T]$, or -1 .
- C. MST Savings: Print total edge weight removable while keeping the graph connected, or -1 . # Contest checklist

State the invariant, choose the model, prove the operation, then code the smallest reliable version.